1. Cel i zakres projektu:

a) Celem tego zadania, było utworzenie programu, który będzie obsługiwał i zarządzał oraz usprawniał pracę hipotetycznej wypożyczalni rowerów.
System korzysta z kolejki(queue), oczekujący klienci oraz listy by śledzić aktualne wypożyczenia.

b) Opis problemu do rozwiązania:
Utworzenie funkcjonalności oraz funkcji, które będą umożliwiały wykonanie poniższych zadań:
   1) Dodawanie osób do kolejki osób, które czekają na rower
   2) Wypożyczanie roweru dla osób z kolejki, zgodnie z kolejnością np. rower będzie wypożyczony pierwszej osobie z kolejki, potem drugiej etc.
   3) Zwrot roweru i usunięcie wypożyczenia z listy, przez co rower, który był poprzednio wypożyczony, może zostać wypożyczony innemu klientowi
   4) Wyświetlanie kolejki osób, które czekają na wypożyczenie oraz listy aktualnie wypożyczonych rowerów
   5) Sortowanie listy aktualnie wypożyczonych rowerów wg. Nazwisk
   6) Zapis i odczyt danych z pliku tekstowego z pomocą biblioteki fstream

c) Uzasadnienie wyboru kontenerów list i queue:
   I. Queue – do kolejki oczekujących, ze względu na FIFO (First In, First Out), prze co idealnie się nadaje do zastosowania jako kolejka przy wypożyczalni rowerów. Operatory jak push (), popback(), które ułatwiają dodawanie elementów na koniec kolejki są tylko dodatkowymi zaletami. Dzięki temu mamy również gwarancje, że użytkownicy będą obsługiwani zgodnie z kolejnością zgłoszeń, co jest efektywne i sprawiedliwe.
   II. List – w wypadku naszego zadania, wypożyczenia mogą być obsługiwane w dowolnej kolejności. Np. Jan Kowalski, może zwrócić rower nr 42 albo Kuba Kostur może zwrócić rower np. 35,
   Z tego powodu wykorzystujemy listę, które pozwala na wykonanie operacji usuwania i wstawiania elementów bez potrzeby przesuwania innych elementów jak przy stosie.

2. Opis zastosowanej struktury danych:

a) Schemat struktury Wypożyczenie:

```
struct Wypożyczenie
{
    std::string nazwisko;
    std::string model;
};
```

Przechowuje ona dane dotyczące pojedynczego wypożyczenia, kto go dokonał(nazwisko), oraz jaki model roweru został wypożyczony(model)

b) Uzasadnienie zastosowania kontenerów STL:

I. Queue – jest wykorzystywane do obsługi kolejki osób, które czekają na wypożyczenie roweru. Zapewnia kolejność FIFO co jest efektywnym i sprawiedliwym mechanizmem obsługi kolejki.

II. List – przechowuje listę aktualnych wypożyczeń.
Pozwala na szybkie dodawanie, usuwanie i sortowanie elementów, co jest niezbędne do dynamicznego i szybkiego zarządzania listą wypożyczeń.

c) Różnice między **list** a **queue**:

- Queue – ma dostęp do tylko pierwszego elementu. Przykładowe operacje push(),pop(),front().
- List – pełny dostęp przy pomocy iteratora. Możliwość dodawania i usuwania elementów w dowolnym miejscu, sortowanie.

3. Fragmenty kodu z omówieniem:

a. Najważniejsze funkcje programu(z komentarzem):

❖ Dodawanie do kolejki:

```
Wypożyczenie wyp;
        std::cout<<"Podaj nazwisko osoby, która chce wypozyczyc rower: ";
        std::cin>>wyp.nazwisko;
        kolejka.push(wyp.nazwisko);
        if(!kolejka.empty())
        {
            std::cout<<"Dodano do kolejki: "<<wyp.nazwisko;
        }
```

Komentarz:

Użytkownik jest pytany o nazwisko osoby, które jest przypisywanie do elementu struktury przy pomocy „**wyp**”, potem to nazwisko jest zapisywane jako szczytowy element kolejki.

❖ Wypożyczenie roweru:

```
if(!kolejka.empty())
        {
        std::string first_element = kolejka.front(); //pobranie
        kolejka.pop(); //usuniecie
        Wypozyczenie wyp;
        wyp.nazwisko = first_element;
        wyp.model = "Standard";
        wypozyczenie.push_back(wyp);
        std::cout<<"Wypozyczono rower dla: "<<first_element<<"\n";
        }
```

Komentarz:

Zmienna first_element pobiera i przechowuje w pamięci zawartość danego elementu kolejki. Potem usuwa ten element, następnie następuje wykorzystywanie struktury Wypozyczenie przy pomocy „wyp” , do której zostaje przypisane nazwisko , które przechowywała zmienna first_element oraz model roweru jakim jest standard.
Potem następuje umieszczenie tych danych do listy wypożyczenie.

❖ Zwrot roweru:

```
❖  std::string szukane_nazwisko;
❖            bool znaleziono = false;
❖            std::cout<<"Jak nazywa się osoba która zwraca
rower: ";
❖            std::cin>>szukane_nazwisko;
❖            for (auto it = wypozyczenie.begin(); it !=
wypozyczenie.end(); ++it)
❖            {
❖                if(it->nazwisko == szukane_nazwisko)
❖                {
❖                    std::cout<<"Znaleziono wypożyczenie
dla: "<<it->nazwisko<<" (model: "<<it->model<<")\n";
❖                    wypozyczenie.erase(it);
❖                    std::cout<<"Rower został zwrocony";
❖                    znaleziono = true;
❖                    break;
❖                }
❖            }
❖            if(!znaleziono)
❖            {
❖                std::cout<<"Brak wypożyczenia dla
nazwiska: "<<szukane_nazwisko;
❖            }
❖            break;
```

Komentarz:
Ta funkcja najpierw pyta o nazwisko osoby, która chce zwrócić rower.
Potem przy pomocy pętli for iteruje przez cala listę do momentu , aż znajdzie dane nazwisko , w przypadku, gdy zostanie odnalezione te nazwisko zostanie one usunięte z listy wypożyczeń, w przypadku, gdy znajduje się kilka osób z tym samym nazwiskiem, będą one usuwane w po kolei.

Wyświetlanie kolejki oczekujących:

```
//lista wypożyczonych rowerow

            std::cout<<"Kolejka oczekujących osob: \n";
            if(kolejka.empty())
            {
                std::cout<<"Aktualnie nie ma oczekujących klientow!!!";
            }
            else
            {
                std::queue<std::string> kopia = kolejka;
                int num =1 ;
                while(!kopia.empty())
                {
                    std::cout<<"Osoba numer: "<<num<<" to: "<<kopia.front()<<"\n";
                    kopia.pop();
                    num++;
                }
            }
```

Komentarz:
Ta funkcja najpierw robi kopie aktualnej kolejki, a potem wyświetla wszystkie elementy tej kolejki oraz usuwa już wyświetlone elementy tej kopii. Korzystamy z tego w ten sposób , by dalej mieć oryginalną kolejkę, na której będziemy mogli wykonywać pozostałe opcję z menu

- Wyświetlanie listy wypożyczonych rowerów

```
  std::cout<<"Lista wypożyczonych rowerow: \n";
          if(wypozyczenie.empty())
          {
           std::cout<<"Lista jest pusta!!!";
          }
          else
          {
           int num=1;
           for(auto it =wypozyczenie.begin(); it != wypozyczenie.end(); ++it)
           {
                        std::cout <<"Rower numer: "<< num <<" wypożyczony dla: " <<
it->nazwisko<<" to: "
                      << " (model: " << it->model << ")\n";                num++;
           }
          }
```

Komentarz:
Na samym początku dokonuje się sprawdzenie czy lista wypożyczonych rowerów nie jest pusta, jeśli jest, użytkownik dostaje stosowny „feedback”, potem następuje wyświetlenie aktualnej listy wypożyczeń.

❖ Sortowanie listy wypozyczeń po nazwisku:

Funkcja porównaj_nazw:

```
bool porownaj_nazw(const Wypozyczenie&a,const Wypozyczenie&b)

{
    return a.nazwisko < b.nazwisko;
}
```

Komentarz:

Ta funkcja pobiera daną 2 dane ze struktury wypożyczenie, a potem porównuje je.

Reszta Sortowania:

```
 if(wypozyczenie.empty())

            {
                std::cout<<"Lista jest pusta!!!";
            }
            else
            {
                wypozyczenie.sort(porownaj_nazw);
                std::cout<<"Lista posortowana wg. nazwisk: ";
               int num =1 ;
               for(auto it = wypozyczenie.begin(); it != wypozyczenie.end(); ++it)
               {
                std::cout<<"\nNumer: "<<num<<" "<<it->nazwisko<<" model: "<<it-
>model;
                num++;
               }
            }
```

Komentarz:

Sort sortuje nazwiska przy pomocy funkcji porównaj_nazw , potem pętla wyświetla wszystkie nazwiska, już posortowane

b. Obsługa menu i iteracji:

❖ Menu:

```
do
{
screen_clear();
std::cout<<"Wybierz dzialanie:\n"
          <<"1\tDodaj osobe do kolejki oczekujacych (queue<string)\n"
          <<"2\tWypozycz rower pierwszej osobie z kolejki > dodanie do listy (list)\n"
          <<"3\tZwroc rower: usuniecie z listy po nazwisku\n"
          <<"4\tWyswietl kolejke oczekujacych\n"
          <<"5\tWyswietl liste wypozyczonych rowerow\n"
          <<"6\tPosortuj liste wypozyczen wg nazwisk\n"
          <<"7\tZapisz liste do piku tekstowego\n"
          <<"8\tWczytaj dane z pliku\n"
          <<"9\tZakoncz program\n"
          <<"Twoj wybor: ";
std::cin>>u_input;
switch(u_input)
{
    case 1: ···
    case 2: ···
    case 3: ···
    case 4: ···
    case 5: ···
    case 6: ···
    default:
        std::cout<<"Podales liczbe spoza zakresu!";
        break;
};
if (u_input != 9) ···
}
 while (u_input!=9);
```

Komentarz:
Menu działa w funkcji do while, która będzie działać do momentu, kiedy ktoś nie poda liczby spoza zakresu , tak długo jak użytkownik będzie podawać dane, która maja swoje odzwierciedlenie w menu program będzie wykonywać stosowne punkty.

❖ Iterator:

```
  for(std::list<Wypozyczenie>::iterator it = wypozyczenie.begin(); it !=
wypozyczenie.end(); ++it)
            {
                out <<"Rower numer: "<<number
                <<" wypożyczony dla: "<<it->nazwisko
                <<" (model: )"<<it->model<<"\n";
            }
```

Komentarz:
Iteracja w liście odbywa się przy pomocy jawnych iteratorów, tak jak jest to przedstawione na fragmencie kodu powyżej

c. Przykład użycia iteratora

- Iterator pozwala na przechodzenie po kolei po wszystkich elementach listy oraz umożliwia dostęp do poszczególnych pól z tej listy:

```
 for(std::list<Wypozyczenie>::iterator it = wypozyczenie.begin(); it !=
wypozyczenie.end(); ++it)
            {
                out <<"Rower numer: "<<number
                <<" wypożyczony dla: "<<it->nazwisko
                <<" (model: )"<<it->model<<"\n";
            }
```

Komentarz:
Lista przechodzi przez wszystkie elementy listy oraz wyświetla kto i co wypożyczył

d. Przykład sortowania/zapisu do pliku

- Sortowanie listy według nazwisko:

Funkcja porównaj_nazw:

```
bool porownaj_nazw(const Wypozyczenie&a,const Wypozyczenie&b)

{
    return a.nazwisko < b.nazwisko;
}
```

Komentarz:

Ta funkcja pobiera daną 2 dane ze struktury wypożyczenie, a potem porównuje je.
Reszta Sortowania:

```
 if(wypozyczenie.empty())

            {
                std::cout<<"Lista jest pusta!!!";
            }
            else
            {
                wypozyczenie.sort(porownaj_nazw);
                std::cout<<"Lista posortowana wg. nazwisk: ";
               int num =1 ;
               for(auto it = wypozyczenie.begin(); it != wypozyczenie.end(); ++it)
               {
                std::cout<<"\nNumer: "<<num<<" "<<it->nazwisko<<" model: "<<it-
>model;
                num++;
               }
            }
```

Komentarz:
Sort sortuje nazwiska przy pomocy funkcji porównaj_nazw , potem pętla wyświetla wszystkie nazwiska, już posortowane

Zapis do pliku:

```
std::ofstream out("dane.txt");

            if(!out.is_open())
            {
                std::cout<<"Nie można otworzyć pliku do zapisu!!!";
            }
            int number =1;
            for(std::list<Wypozyczenie>::iterator it = wypozyczenie.begin(); it !=
wypozyczenie.end(); ++it)
            {
                out <<"Rower numer: "<<number
                <<" wypożyczony dla: "<<it->nazwisko
                <<" (model: )"<<it->model<<"\n";
            }
            std::cout<<"Dane zostały przypisane do pliku";
```

Komentarz:

Ten fragment kodu dokonuje eksportu danych pod nazwą dane.txt, najpierw pobiera dane przy pomocy pętli for ze struktury wypożyczenie, a potem dokonuje eksportu dla każdego wypożyczenia.

4. Wyniki działania programu:

a. Zrzuty ekranu z działania programu(terminal):

1) Opcja numer1:

```
Wybierz dzialanie:
1       Dodaj osobe do kolejki oczekujacych (queue<string)
2       Wypozycz rower pierwszej osobie z kolejki > dodanie do listy (list)
5       Wyswietl liste wypozyczonych rowerow
6       Posortuj liste wypozyczen wg nazwisk
7       Zapisz liste do piku tekstowego
8       Wczytaj dane z pliku
9       Zakoncz program
Twoj wybor: 1
Podaj nazwisko osoby, ktora chce wypozyczyc rower: Kowal
Dodano do kolejki: Kowal
Nacisnij enter by kontynuowac....
```

2) Opcja numer2:

```
8       Wczytaj dane z pliku
9       Zakoncz program
Twoj wybor: 2
Wypozyczono rower dla: Kowal

Nacisnij enter by kontynuowac....
```

3) Opcja numer3:

```
9       Zakoncz program
Twoj wybor: 3
Jak nazywa sie osoba ktora zwraca rower: Kowal
Znaleziono wypozyczenie dla: Kowal (model: Standard)
Rower zostal zwrocony
Nacisnij enter by kontynuowac....
```

4) Opcja numer4(dodałem 5 przykładowych danych):

```
9       Zakoncz program
Twoj wybor: 4
Kolejka oczekujacych osob:
Osoba numer: 1 to: Wayne
Osoba numer: 2 to: Parker
Osoba numer: 3 to: Bateman
Osoba numer: 4 to: Schwarz
Osoba numer: 5 to: Bertman

Nacisnij enter by kontynuowac.
```

5) Opcja numer5(wypożyczyłem rowery tamtym 5 użytkownikom):

```
Twoj wybor: 5
Lista wypozyczonych rowerow:
Rower numer: 1 wypozyczony dla: Wayne to:  (model: Standard)
Rower numer: 2 wypozyczony dla: Parker to:  (model: Standard)
Rower numer: 3 wypozyczony dla: Bateman to:  (model: Standard)
Rower numer: 4 wypozyczony dla: Schwarz to:  (model: Standard)
Rower numer: 5 wypozyczony dla: Bertman to:  (model: Standard)
```

6) Opcja numer6:

```
9       Zakoncz program
Twoj wybor: 6
Lista posortowana wg. nazwisk:
Numer: 1 Bateman model: Standard
Numer: 2 Bertman model: Standard
Numer: 3 Parker model: Standard
Numer: 4 Schwarz model: Standard
Numer: 5 Wayne model: Standard
Nacisnij enter by kontynuowac....
```

7) Opcja numer7:

```
Twoj wybor: 7
Dane zostaly przypisane do pliku
```

8) 8.Opcja numer8(zrestartowałem program):

```
9       Zakoncz program
Twoj wybor: 8
Dane zostaly wczytane z pliku kowalsi.txt!!!
Nacisnij enter by kontynuowac....
```

9) Opcja numer5:

```
8       Wczytaj dane z pliku
Rower numer: 44 wypozyczony dla: Wojcik to:  (model: Miejski)
Rower numer: 45 wypozyczony dla: Walczak to:  (model: Sportowy)
Rower numer: 46 wypozyczony dla: Jankowski to:  (model: Sportowy)
Rower numer: 47 wypozyczony dla: Wrobel to:  (model: Elektryczny)
Rower numer: 48 wypozyczony dla: Nowak to:  (model: Sportowy)
Rower numer: 49 wypozyczony dla: Walczak to:  (model: Miejski)
Rower numer: 50 wypozyczony dla: Wisniewski to:  (model: Miejski)
```

b. Przykładowy plik.txt po eksporcie danych:
Rower numer: 1 wypożyczony dla: Bateman (model:)Standard
Rower numer: 1 wypożyczony dla: Bertman (model:)Standard
Rower numer: 1 wypożyczony dla: Parker (model:)Standard
Rower numer: 1 wypożyczony dla: Schwarz (model:)Standard
Rower numer: 1 wypożyczony dla: Wayne (model:)Standard

5. Wnioski końcowe:

a. Zalety i ograniczenia rozwiązania,
   Zalety:
   - Queue i list pozwoliły na efektywne zarządzanie i małą i dużą ilością danych oraz znacznie usprawniły samą implementacje odpowiedniego rozwiązania.
   - Queue – idealnie się nadało do zrealizowania zadania kolejki oczekujących
   - List – umożliwiło dynamiczną i sprawną administracje wypożyczeniami , w tym dodawanie usuwanie i sortowanie elementów
   - Dzięki powyższym rozwiązaniom, program ma bardzo łatwą dalszą ścieżkę rozwoju i dalszej modernizacji, zgodnie z obranym kierunkiem.
   - Zapis i odczyt danych z pliku txt pozwala na przechowywanie danych i szybki import tych danych, i na dłuższą metę pozwala ograniczyć koszta ciągłego działania programu, dzięki takiemu rozwiązaniu wystarczy program włączyć, kiedy jest potrzebny

   Ograniczenia:

   - Podczas kiedy do prostej implementacji jak w przypadku tego zadania, te rozwiązania nadzą się idealnie to w przypadku rozwiązania komercyjnego, bardzo szybko bylibyśmy przytłoczeni natłokiem problemów do rozwiązania np. przechowywaniem danych osobowych w plikach , które musiałyby być „zahashowane”, co utrudnia korzystanie np. z biblioteki fstream
   - Założenia zadania korzystają z bardzo prostych założeń , bez np. dat wypożyczenia, kosztów samej usługi , kar za nieoddanie roweru w terminie etc.
   - Brak obsługi multithreadingu lub kilku użytkowników jednocześnie
b. Propozycje dalszego rozwoju programu:
   - Utworzenie interfejsu GUI , w celu uprzyjemnienia procesu zapisu danych
   - Zrezygnowanie z zapisu danych osobowych do pliku.txt i zmodyfikowanie programu, by zapis następował do bazy SQL’owej
   - Rozbudowanie struktury wypożyczenie o kolejne pola jak ID roweru, czas wypożyczenia, kaucja etc.
   - Przy pomocy bazy danych dodanie raportowanie oraz statystyk, co pomoże w dalszej pracy „data analyst” w celu wyłonienia np. najczęściej wypożyczanych rowerów i np. w celu utworzenia promocji/okazji , by zachęcić użytkowników do spróbowania np. innych typów rowerów
c. Refleksja nad wykorzystaniem STL w praktyce:
   Gotowe rozwiązania jakim są kolejki i listy, znacząco ułatwiło pracę przy tym programie. Szybka i efektywna praca, pokazała ,że tego typu podejście do tego problemu, mogłoby zadziałać w prawdziwej wypożyczalni.
   Wygodne mechanizmy, iteracji, sortowania i manipulacji danymi pozwoliło skupić

się na kodowaniu zamiast na tworzeniu struktur od zera. Z kolei należy również napomnieć ,że ten program i dalsza analiza nad tym co można by poprawić w tym programie wykazała również, że tak naprawdę komercjalne projekty i rozwiązania wymagają elastyczności i gotowości na szukanie nowych rozwiązań.